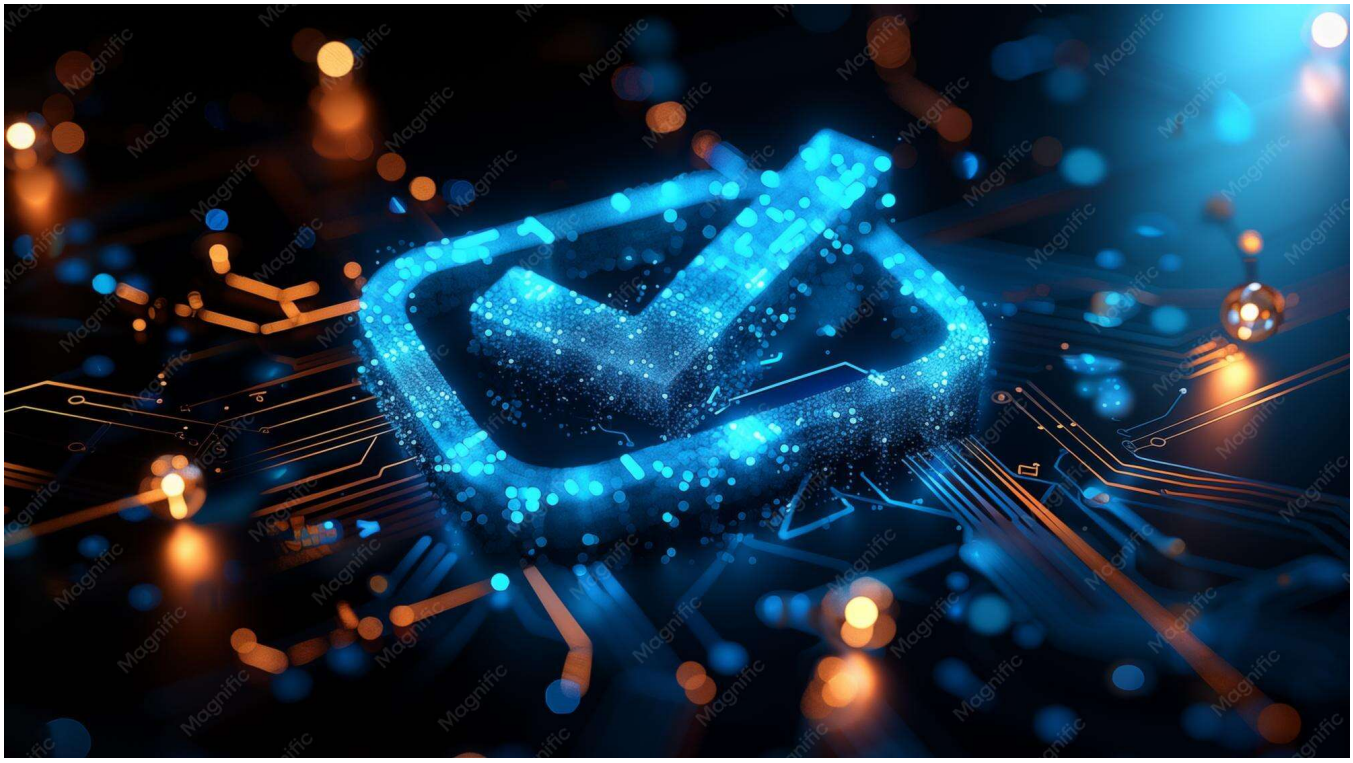


Project: UVM FIFO

Synchronous FIFO - Full UVM Verification Environment

Name: Ahmed Wael Sadek El Sawy

Project: UVM Synchronous FIFO

1. Verification Plan

Each requirement of the FIFO has a label (FIFO_1 to FIFO_12). For every label the plan states how the stimulus is generated by the UVM sequences, which coverage proves it was exercised, and which checker (assertion, scoreboard) proves it is correct.

Label	Design Requirement Description	Stimulus Generation	UVM Sequence(s)	Functional Coverage	Functionality Check
FIFO_1	When the reset is asserted (rst_n = 0), the FIFO must reset asynchronously. The internal write pointer, read pointer and counter must be 0, wr_ack/overflow/underflow must be 0 and the FIFO must be considered empty.	fifo_reset_seq drives rst_n = 0 at the start of the simulation (driven on the falling clock edge, so it is asynchronous to the clock). fifo_write_read_seq random phase does not force rst_n, so reset_c asserts it at random (5%) while the FIFO holds data.	fifo_reset_seq; fifo_write_read_seq (random phase)	Reset items in the random phase fill the cross_empty bins with wr_en = 1 (reset requested while a write is requested).	Reset assertions a_reset_count, a_reset_wr_ptr, a_reset_rd_ptr, a_reset_flags. Scoreboard clears its reference queue when rst_n = 0.
FIFO_2	When wr_en is asserted and the FIFO is not full, the input data must be written successfully and wr_ack must be asserted.	fifo_write_only_seq: 15 back-to-back writes with random data_in (rst_n = 1, wr_en = 1, rd_en = 0). Random phase: wr_en is high 70% of the time.	fifo_write_only_seq; fifo_write_read_seq	wr_en_cp, wr_ack_cp, cross_wr_ack (wr_en x rd_en x wr_ack).	p_wr_ack. Scoreboard stores the accepted word in mem_ref and later compares it with data_out.
FIFO_3	When a write is attempted while the FIFO is full, the write must be rejected, overflow must be asserted, wr_ack must stay 0 and the FIFO contents must not be corrupted.	fifo_write_only_seq keeps writing after the 8th word (items 9 to 15 are attempts on a full FIFO). The random phase adds more attempts.	fifo_write_only_seq; fifo_write_read_seq	overflow_cp, cross_overflow, cross_full (wr_en = 1 with full = 1).	p_overflow, p_no_ack_when_full. Scoreboard does not push when mem_ref is full, so later reads expose any corruption.
FIFO_4	When a read is attempted while the FIFO is empty, the read must be rejected, underflow must be asserted and the FIFO contents must not be corrupted.	fifo_read_only_seq keeps reading after the 8th word (items 9 to 15 are attempts on an empty FIFO). fifo_write_read_seq starts at an empty FIFO with wr_en = rd_en = 1.	fifo_read_only_seq; fifo_write_read_seq	underflow_cp, cross_underflow, cross_empty (rd_en = 1 with empty = 1).	p_underflow. Scoreboard does not pop when mem_ref is empty.
FIFO_5	When the internal count is zero the empty flag must be asserted. The FIFO enters the empty state after the last valid entry is read.	fifo_read_only_seq drains the full FIFO to count = 0. Random reads and writes revisit the empty state many times.	fifo_read_only_seq; fifo_write_read_seq	empty_cp, cross_empty (all wr_en x rd_en combinations with empty = 1).	p_empty. Cover directive p_empty proves the state was reached.
FIFO_6	When the internal count equals FIFO_DEPTH the full flag must be asserted. The FIFO enters the full state after FIFO_DEPTH valid writes.	fifo_write_only_seq fills the FIFO from empty to full (8th write).	fifo_write_only_seq; fifo_write_read_seq	full_cp, cross_full (all valid wr_en x rd_en combinations with full = 1).	p_full. Cover directive p_full.
FIFO_7	When the internal count reaches FIFO_DEPTH - 1 the almostfull flag must be asserted (only one more write can be performed before the FIFO is full).	fifo_write_only_seq passes through count = 7 on the 7th write. Random phase revisits it.	fifo_write_only_seq; fifo_write_read_seq	almostfull_cp, cross_almostfull (wr_en x rd_en x almostfull).	p_almostfull. Cover directive p_almostfull.
FIFO_8	When the internal count equals 1 the almostempty flag must be asserted (only one more read can be performed before the FIFO is empty).	fifo_read_only_seq passes through count = 1 on the 7th read. fifo_write_read_seq (50 simultaneous items) keeps count = 1 for a long time.	fifo_read_only_seq; fifo_write_read_seq	almostempty_cp, cross_almostempty (wr_en x rd_en x almostempty).	p_almostempty. Cover directive p_almostempty.

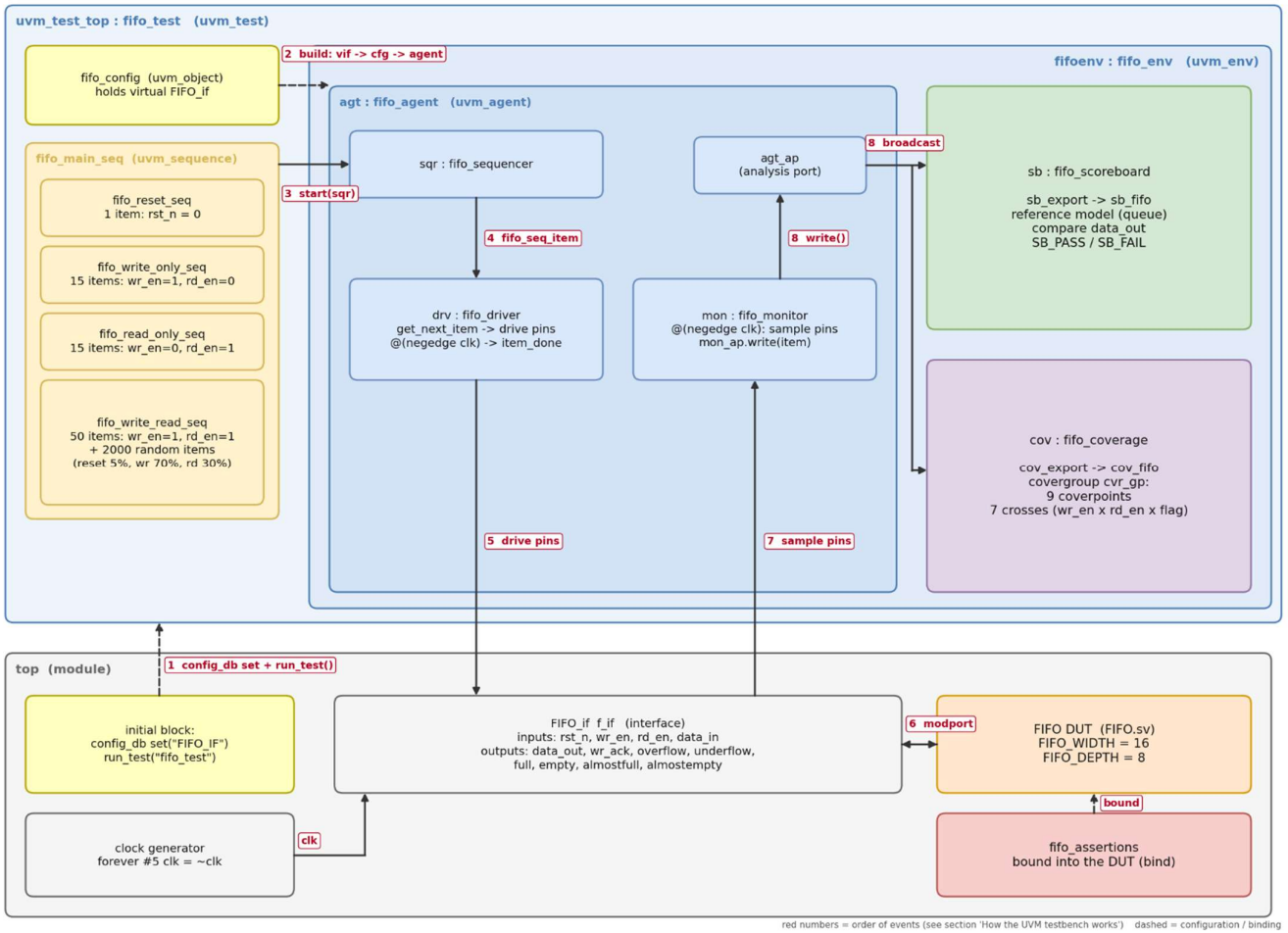
Project: UVM Synchronous FIFO

Label	Design Requirement Description	Stimulus Generation	UVM Sequence(s)	Functional Coverage	Functionality Check
FIFO_9	After FIFO_DEPTH valid writes or reads, the corresponding write or read pointer must wrap around from FIFO_DEPTH - 1 back to 0.	Writes wrap in fifo_write_only_seq (8th write); reads wrap in fifo_read_only_seq (8th read); the 50 simultaneous items wrap both pointers repeatedly.	fifo_write_only_seq; fifo_read_only_seq; fifo_write_read_seq	Cover directives p_wr_wrap and p_rd_wrap (sequential domain coverage).	p_wr_wrap, p_rd_wrap. Scoreboard checks FIFO order across the wrap.
FIFO_10	The internal write pointer, read pointer and counter must never exceed the valid FIFO range at any time.	Long randomized run (2000 items) with 5% reset, 70% writes and 30% reads, so the FIFO moves through all occupancy levels.	fifo_write_read_seq (random phase)	Occupancy extremes: empty_cp, almostempty_cp, almostfull_cp, full_cp. Cover directives p_empty to p_full.	p_threshold (count <= FIFO_DEPTH, wr_ptr < FIFO_DEPTH, rd_ptr < FIFO_DEPTH).
FIFO_11	When wr_en and rd_en are both high the FIFO must follow the specification: when empty only the write takes place; when full only the read takes place; otherwise both operations take place.	fifo_write_read_seq: 50 items with wr_en = rd_en = 1 starting from an empty FIFO, then random items that reach full and intermediate occupancy.	fifo_write_read_seq	wr_en = 1, rd_en = 1 bins of cross_empty, cross_full, cross_almostempty and cross_almostfull.	p_wr_rd_empty, p_wr_rd_full, p_wr_rd_mid (count update rules). Scoreboard applies the same push/pop rules.
FIFO_12	The sequential data_out must correspond to the correct FIFO entry on a valid read, preserving first-in, first-out order.	Every sequence uses randomize() for data_in, so every word is different and an ordering error is easy to detect.	All sequences	wr_en_cp and rd_en_cp (every read and write is observed).	Scoreboard: reference queue mem_ref, expected value = mem_ref.pop_front(), compared with data_out (SB_PASS / SB_FAIL, summary in report_phase).

Project: UVM Synchronous FIFO

2. UVM Testbench Architecture

The figure shows the UVM structure of the project. The red numbers give the order of events and are explained one by one in section 3. Dashed arrows are configuration / binding, solid arrows are data flow.



3. How the UVM Testbench Works

This section follows one transaction from the top module to the report. Think of the testbench as a restaurant: the **sequence** writes the orders, the **sequencer** is the queue, the **driver** is the waiter that puts the order on the interface, the **DUT** is the kitchen, the **monitor** watches what goes in and comes out, the **scoreboard** is the inspector with the recipe book, and the **coverage collector** counts which dishes were tried.

3.1 The flow in ten steps (numbers match the diagram)

1. **Top module** creates the clock, the interface and the DUT, binds the assertions, stores the interface in `uvm_config_db` and calls `run_test()`.
2. **Build phase** (zero time): the test creates the environment, the configuration object and the main sequence; the interface is taken from `uvm_config_db` and handed to the agent through the config object.
3. **Connect phase**: driver to sequencer, monitor to analysis port, agent port to scoreboard and coverage collector.
4. **Run phase starts**: the test raises an objection and starts `fifo_main_seq` on the sequencer.
5. **Sequences** create `fifo_seq_item` objects and randomize them under constraints.
6. **Driver** takes each item from the sequencer and drives the interface pins.
7. **DUT** reacts on the clock edge and changes its outputs.
8. **Monitor** samples inputs and outputs from the interface and broadcasts them as a transaction.
9. **Analysis**: scoreboard compares with its reference model, the covergroup is sampled, assertions check in parallel.
10. **End of test**: the objection is dropped, the report phase prints the summary and the simulation finishes.

3.2 Step 1 - The top module (top.sv)

`top` is the only hardware module of the testbench. It generates the clock (period 10 ns), instantiates the interface `f_if` and the DUT, and binds `fifo_assertions` into the FIFO so the assertions can see the interface and the internal `count`, `wr_ptr` and `rd_ptr`.

The class-based UVM world cannot reach module signals directly, so the interface is passed as a **virtual interface** through `uvm_config_db` under the key `"FIFO_IF"`. `run_test("fifo_test")` then creates the test and starts the UVM phases.

3.3 Step 2 - Build phase

UVM builds the hierarchy top-down. `fifo_test` creates `fifoenv`, `fifo_cfg` and `main_seq`, then calls `uvm_config_db::get(this,"","FIFO_IF",fifo_cfg.fifo_vif)`. It publishes the config object with `uvm_config_db::set(this,"*","CFG",fifo_cfg)`. The environment creates the agent, scoreboard and coverage collector; the agent reads `"CFG"` and creates the sequencer, driver and monitor. All objects are created with the **factory** (`type_id::create`), which allows a class to be replaced later without editing its parent.

The keys must match exactly on both sides. A typo in `"FIFO_IF"` or `"CFG"` gives the fatal message *unable to get*; in QuestaSim the UVM Details window (ConfigDB view) shows the write and read count of every key.

3.4 Step 3 - Connect phase

Connection (written in)	Meaning
agent: drv.fifo_vif = fifo_cfg.fifo_vif; mon.fifo_vif = fifo_cfg.fifo_vif	driver and monitor receive the virtual interface (the only classes that touch the pins)
agent: drv.seq_item_port.connect(sqr.seq_item_export)	the driver pulls items from the sequencer
agent: mon.mon_ap.connect(agt_ap)	the monitor's analysis port is forwarded to the agent's port
env: agt.agt_ap.connect(sb.sb_export)	every monitored transaction goes to the scoreboard
env: agt.agt_ap.connect(cov.cov_export)	every monitored transaction goes to the coverage collector
scoreboard and coverage: export.connect(fifo.analysis_export)	each has a `uvm_tlm_analysis_fifo` that stores incoming items until its run_phase thread calls `get()`

3.5 Step 4 - Run phase and objections

The run phase is the only phase that consumes simulation time, and all components run in parallel. `fifo_test` raises an objection ("do not end yet"), starts `main_seq` on `fifoenv.agt.sqr`, and drops the objection when the sequence returns. When every objection is dropped UVM ends the run phase, executes the report phase and finishes.

3.6 Step 5 - Sequences and the sequence item

`fifo_main_seq` starts four sub-sequences one after the other: `fifo_reset_seq`, `fifo_write_only_seq`, `fifo_read_only_seq` and `fifo_write_read_seq`. Each one is a separate class in its own file and package. The constraints are in the item class `fifo_seq_item`:

- `reset_c`: `rst_n dist {0 := 5, 1 := 95}` - reset is rare.
- `wr_en_c`: write enable is high 70% of the time; `rd_en_c`: read enable is high 30% of the time (variables `WR_EN_ON_DIST`, `RD_EN_ON_DIST`).
- The directed sequences add inline constraints, for example `randomize() with { rst_n == 1; wr_en == 1; rd_en == 0; }` in the write-only sequence.

The handshake for every item is: `start_item(req)` asks the sequencer for a turn; `randomize()` chooses the values; `finish_item(req)` hands the item over and **blocks**; the driver's `get_next_item()` receives it; the driver drives the pins; `item_done()` releases `finish_item`, and the sequence builds the next item.

3.7 Step 6 - Driving the interface (fifo_driver)

The driver loops forever: `get_next_item(req)`, write `rst_n`, `wr_en`, `rd_en` and `data_in` into the virtual interface, wait `@(negedge fifo_vif.clk)`, then `item_done()`. Inputs therefore change on the **falling** clock edge, half a period before the rising edge that the DUT uses, so there are no race conditions. One item takes exactly one clock period (10 ns): item k is driven at 10 x k ns. Because the reset is driven on the falling edge it acts as an **asynchronous** reset relative to the rising clock edge: the outputs clear immediately.

3.8 Step 7 - What the DUT does

On each rising edge the FIFO writes `mem[wr_ptr]` when `wr_en` and not full, reads `mem[rd_ptr]` when `rd_en` and not empty, and updates `count`. `wr_ack`, `overflow`, `underflow` and `data_out` are **registered** outputs, so they show the result of an operation one clock edge after the inputs were

applied. The flags `full`, `empty`, `almostfull` and `almostempty` are combinational functions of `count`.

3.9 Step 8 - Monitoring (fifo_monitor)

The monitor samples on the falling edge, **before** the driver applies the next item at the same instant. At that moment the inputs are the item driven half a period earlier and the outputs already contain its result. It copies all twelve interface signals into a new `fifo_seq_item` and calls `mon_ap.write(item)`. The analysis port is a broadcast: UVM delivers the same item to the scoreboard and to the coverage collector.

Time	Driver (falling edge)	DUT (rising edge)	Monitor (falling edge)
0 ns	applies item 0: rst_n = 0	asynchronous reset at once: wr_ack = overflow = underflow = 0, count = 0, empty = 1	-
10 ns	applies item 1: rst_n = 1, wr_en = 1, data_in = D1	-	samples item 0 (reset values); scoreboard clears its queue
15 ns	-	mem[0] = D1, wr_ptr = 1, count = 1, wr_ack = 1, empty = 0, almostempty = 1	-
20 ns	applies item 2 (next write)	-	samples item 1: wr_en = 1, wr_ack = 1, almostempty = 1; scoreboard pushes D1; covergroup sampled

3.10 Step 9 - Analysis of the output

Scoreboard (fifo_scoreboard) - reference model

The scoreboard keeps a queue `mem_ref`. For every item: if `rst_n = 0` the queue is cleared. Otherwise it decides what the DUT must have done: with `wr_en` and `rd_en` together, an empty FIFO only writes, a full FIFO only reads, anything else does both; with one enable alone the operation happens only if the FIFO is not full (write) or not empty (read). A write pushes `data_in`; a read pops the oldest word, and the popped value must equal the monitored `data_out`. A difference raises `uvm_error("SB_FAIL")`. `report_phase` prints the number of correct and wrong comparisons.

Coverage collector (fifo_coverage)

Each monitored item is sampled in the covergroup `cvr_gp`: nine coverpoints (wr_en, rd_en, full, empty, almostfull, almostempty, overflow, underflow, wr_ack) and seven crosses of wr_en x rd_en with each flag. Bins that can never happen are excluded with `ignore_bins` (section 7).

Assertions (fifo_assertions)

The assertions are not part of the UVM class hierarchy. They are bound into the DUT and check every clock edge for the whole simulation, for example `(wr_en && full) |=> overflow`. Scoreboard and assertions complement each other: the scoreboard checks the data path (order and values), the assertions check flags, counters and timing.

3.11 Step 10 - End of the test

When `main_seq` finishes, the test drops its objection, UVM stops the run phase, the scoreboard prints its summary in `report_phase`, UVM prints the report summary (counts of UVM_INFO/WARNING/ERROR/FATAL) and calls `$finish`. QuestaSim reports this as *Break in Task uvm_pkg::uvm_root::run_test*; the simulation is still loaded, so the waveform can be inspected and `save_cov.do` can save the coverage.

Project: UVM Synchronous FIFO

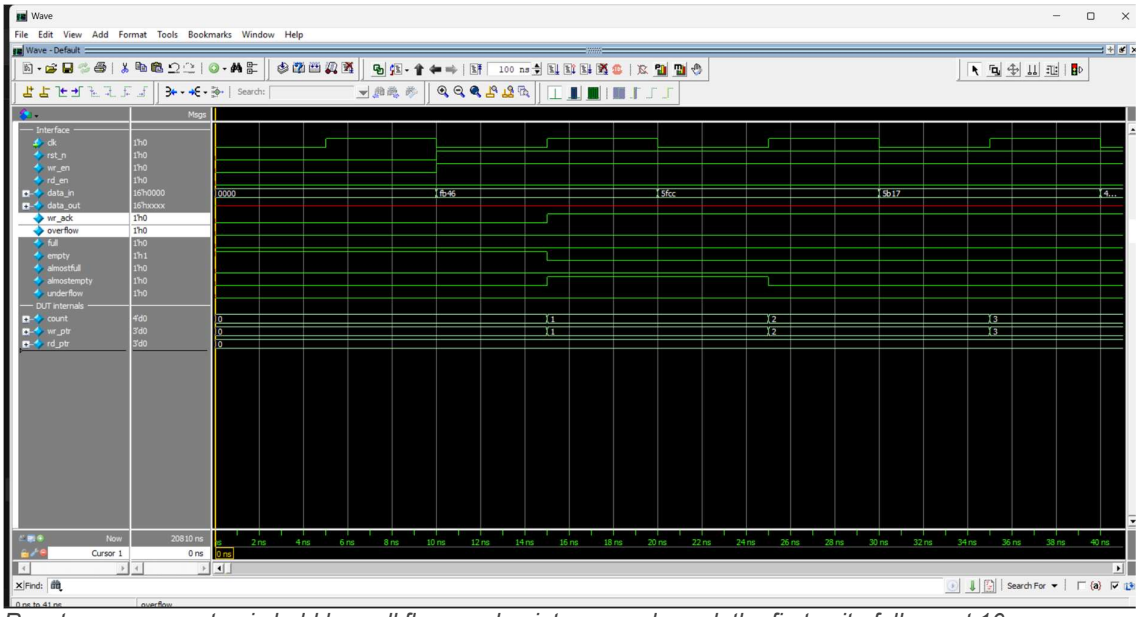
4. Assertions Table

Plan label	Feature	Assertion	Name / type
FIFO_1	When rst_n = 0 the counter is 0	always_comb if (!rst_n) a_reset_count: assert final (count == 0)	Deferred immediate (assert final)
FIFO_1	When rst_n = 0 the write pointer is 0	always_comb if (!rst_n) a_reset_wr_ptr: assert final (wr_ptr == 0)	Deferred immediate (assert final)
FIFO_1	When rst_n = 0 the read pointer is 0	always_comb if (!rst_n) a_reset_rd_ptr: assert final (rd_ptr == 0)	Deferred immediate (assert final)
FIFO_1	When rst_n = 0 wr_ack, overflow and underflow are 0	always_comb if (!rst_n) a_reset_flags: assert final (!wr_ack && !overflow && !underflow)	Deferred immediate (assert final)
FIFO_2	A write to a non-full FIFO is acknowledged	@(posedge clk) disable iff (!rst_n) (wr_en && !full) |=> wr_ack	p_wr_ack
FIFO_3	A write to a full FIFO is not acknowledged	@(posedge clk) disable iff (!rst_n) (wr_en && full) |=> !wr_ack	p_no_ack_when_full
FIFO_3	A write to a full FIFO raises overflow	@(posedge clk) disable iff (!rst_n) (wr_en && full) |=> overflow	p_overflow
FIFO_4	A read from an empty FIFO raises underflow	@(posedge clk) disable iff (!rst_n) (rd_en && empty) |=> underflow	p_underflow
FIFO_5	count = 0 means empty	@(posedge clk) disable iff (!rst_n) (count == 0) |-> empty	p_empty
FIFO_6	count = FIFO_DEPTH means full	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH) |-> full	p_full
FIFO_7	count = FIFO_DEPTH-1 means almostfull	@(posedge clk) disable iff (!rst_n) (count == FIFO_DEPTH-1) |-> almostfull	p_almostfull
FIFO_8	count = 1 means almostempty	@(posedge clk) disable iff (!rst_n) (count == 1) |-> almostempty	p_almostempty
FIFO_9	Write pointer wraps from FIFO_DEPTH-1 to 0	@(posedge clk) disable iff (!rst_n) (wr_ptr == FIFO_DEPTH-1 && wr_en && !full) |=> (wr_ptr == 0)	p_wr_wrap
FIFO_9	Read pointer wraps from FIFO_DEPTH-1 to 0	@(posedge clk) disable iff (!rst_n) (rd_ptr == FIFO_DEPTH-1 && rd_en && !empty) |=> (rd_ptr == 0)	p_rd_wrap
FIFO_10	Counter and pointers never exceed the valid range	@(posedge clk) disable iff (!rst_n) (count <= FIFO_DEPTH) && (wr_ptr < FIFO_DEPTH) && (rd_ptr < FIFO_DEPTH)	p_threshold
FIFO_11	wr_en and rd_en together on an empty FIFO: only the write happens (count + 1)	@(posedge clk) disable iff (!rst_n) (wr_en && rd_en && empty) |=> (count == $past(count) + 1)	p_wr_rd_empty
FIFO_11	wr_en and rd_en together on a full FIFO: only the read happens (count - 1)	@(posedge clk) disable iff (!rst_n) (wr_en && rd_en && full) |=> (count == $past(count) - 1)	p_wr_rd_full
FIFO_11	wr_en and rd_en together on a partly filled FIFO: both happen (count unchanged)	@(posedge clk) disable iff (!rst_n) (wr_en && rd_en && !empty && !full) |=> (count == $past(count))	p_wr_rd_mid

5. UVM Sequences and Waveform Snippets

5.1 fifo_reset_seq

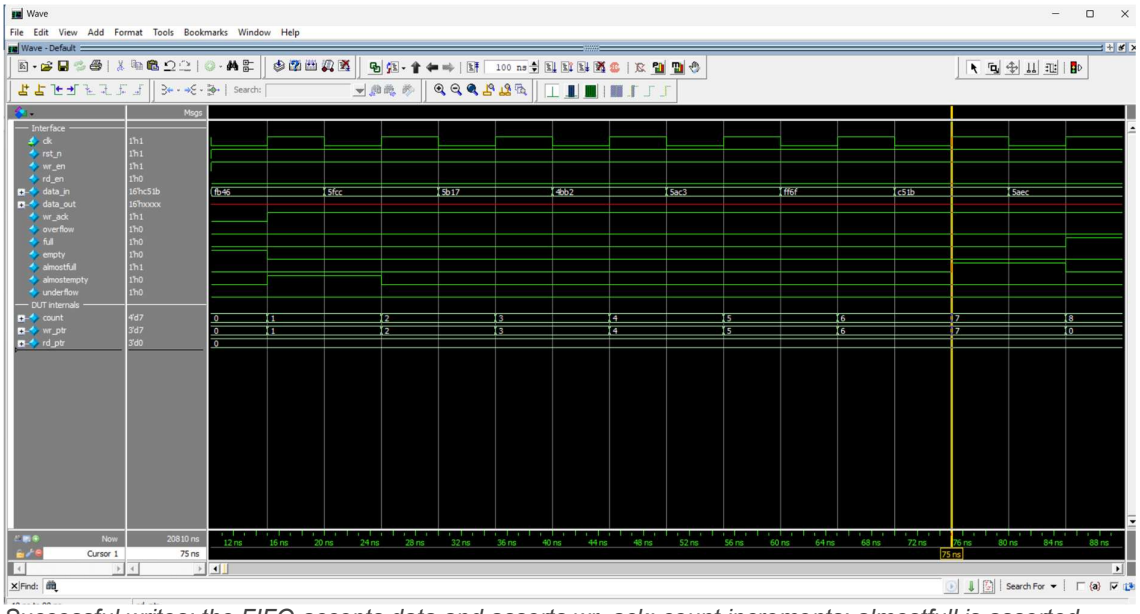
Reset sequence



Reset sequence: rst_n is held low, all flags and pointers are cleared; the first write follows at 10 ns.

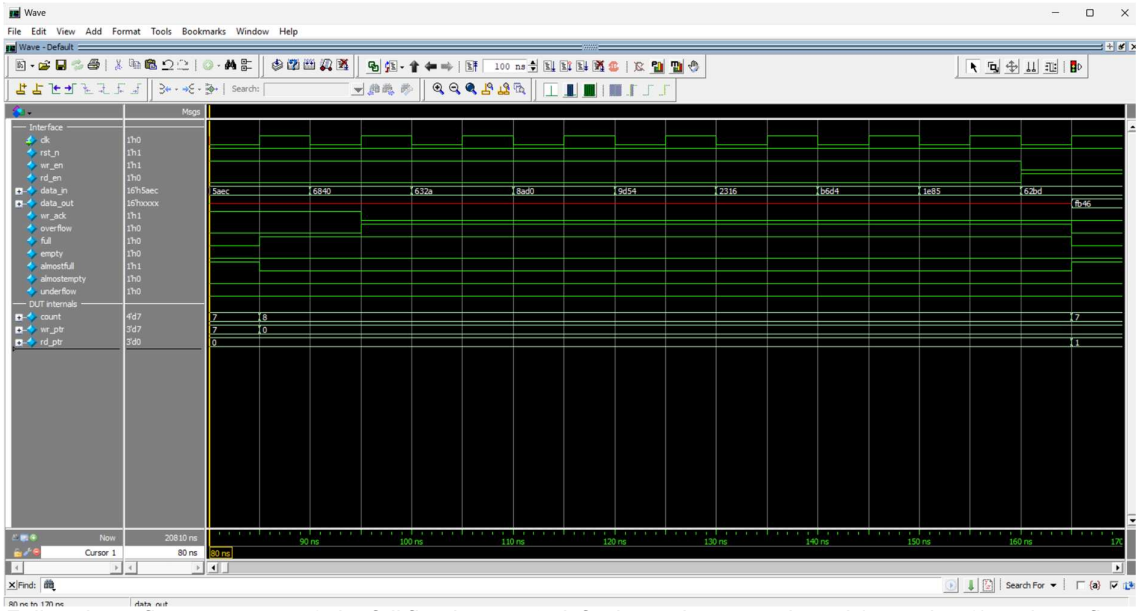
5.2 fifo_write_only_seq

Normal successful writes and almost full



Successful writes: the FIFO accepts data and asserts wr_ack; count increments; almostfull is asserted when count = 7.

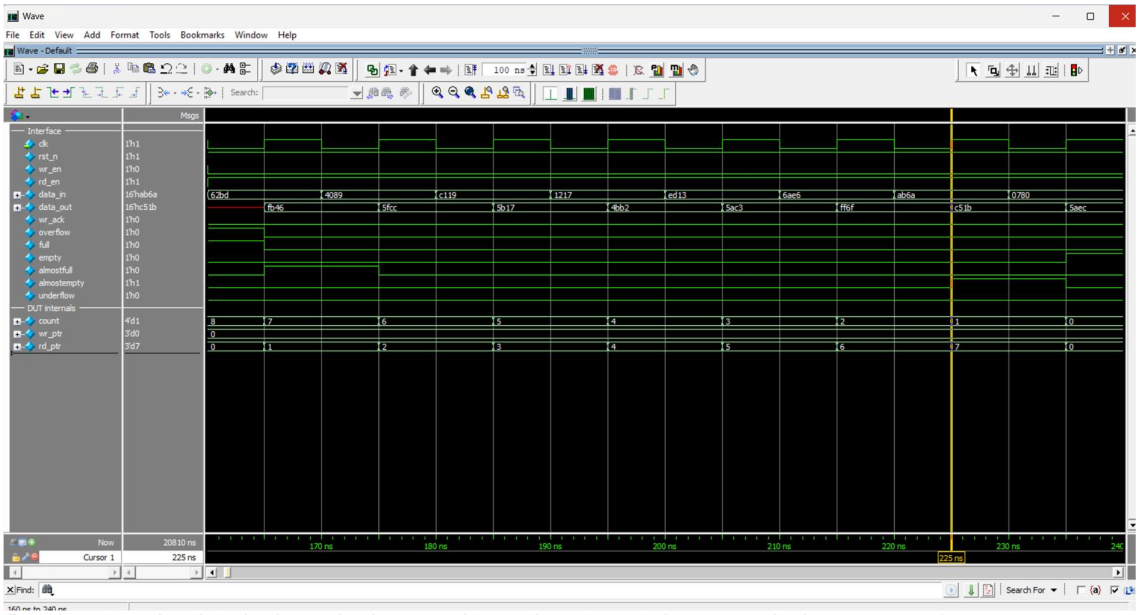
Full + overflow and write-pointer wraparound



Full and overflow: at count = 8 the full flag is asserted; further writes are rejected (wr_ack = 0) and overflow = 1.

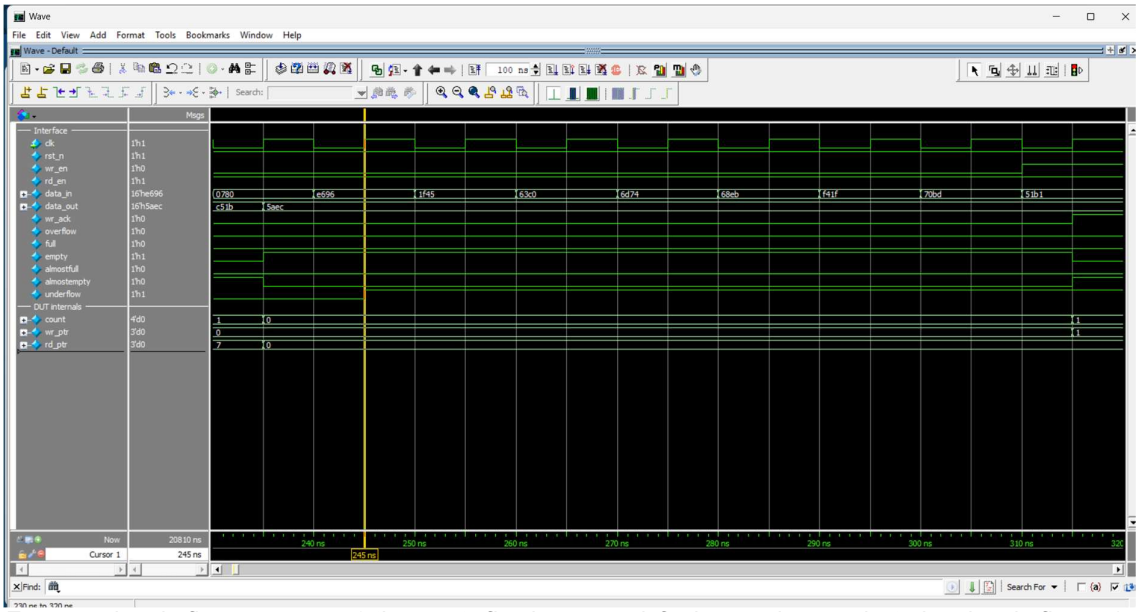
5.3 fifo_read_only_seq

Reads and almost empty



Reads return the data in the order it was written; almostempty is asserted when count = 1.

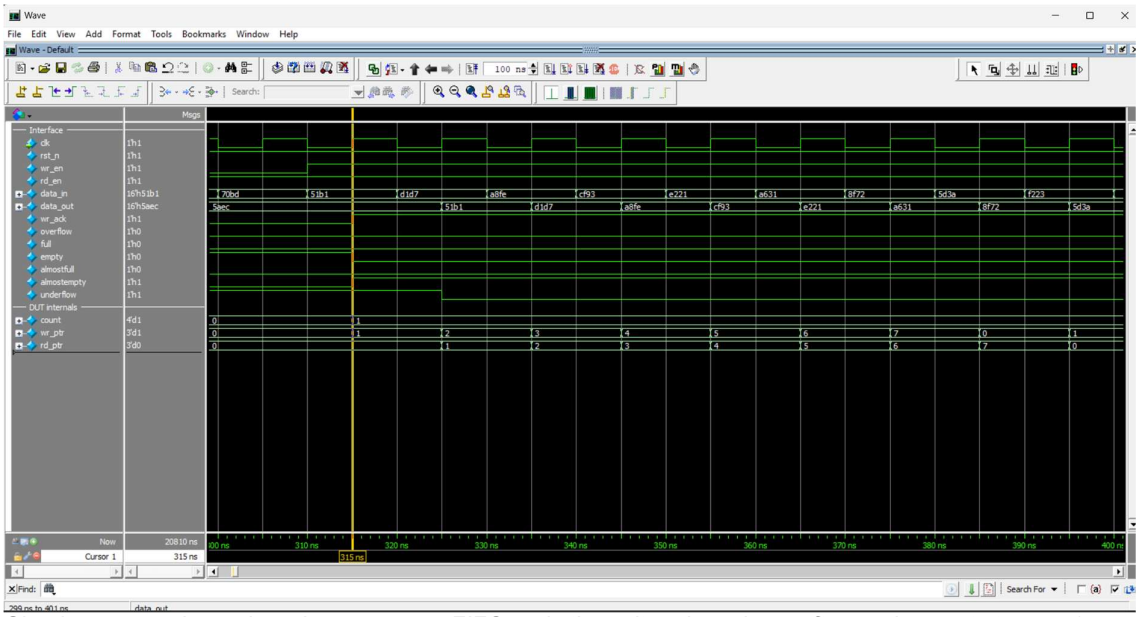
Empty + underflow and read-pointer wraparound



Empty and underflow: at count = 0 the empty flag is asserted; further reads are rejected and underflow = 1.

5.4 fifo_write_read_seq - simultaneous part

Simultaneous wr_en = 1 and rd_en = 1 - FIFO empty

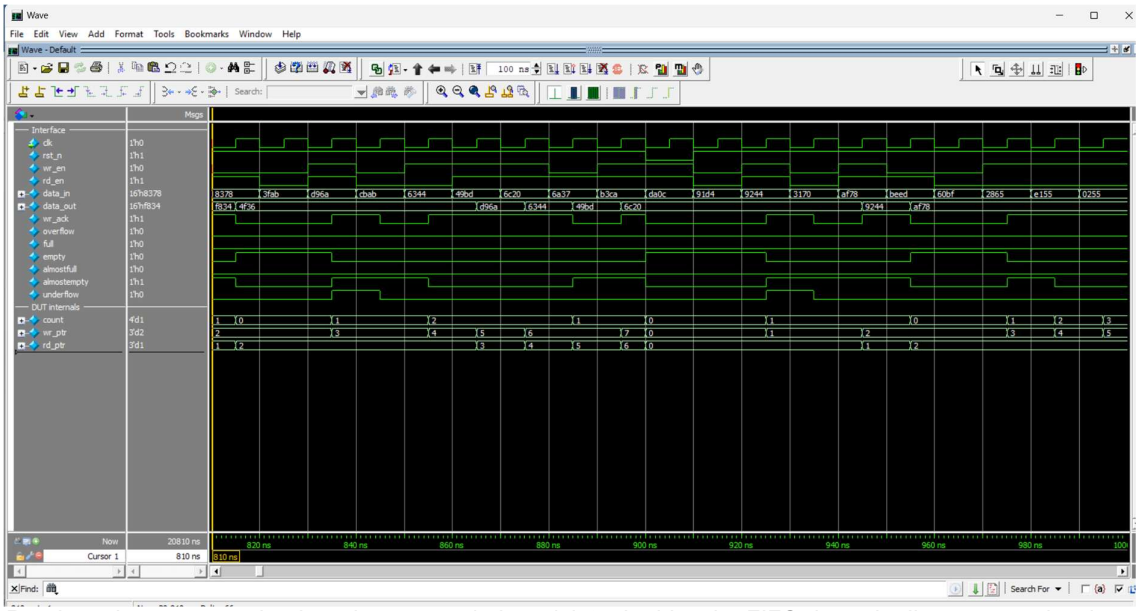


Simultaneous write and read on an empty FIFO: only the write takes place; afterwards count stays at 1 while data flows through.

5.5 fifo_write_read_seq - random part

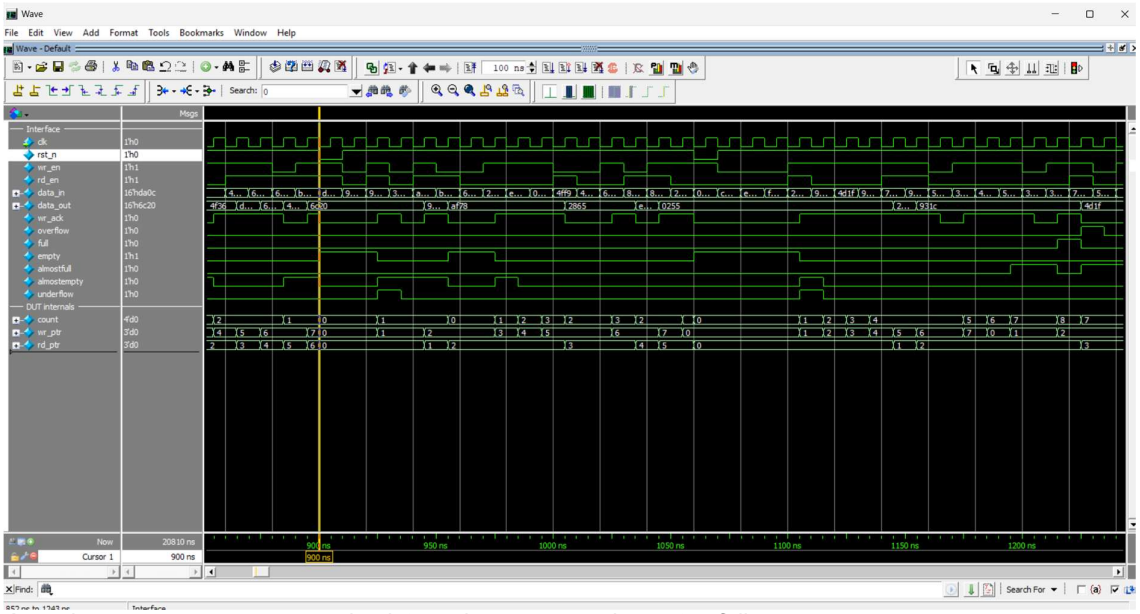
Project: UVM Synchronous FIFO

Random stimulus



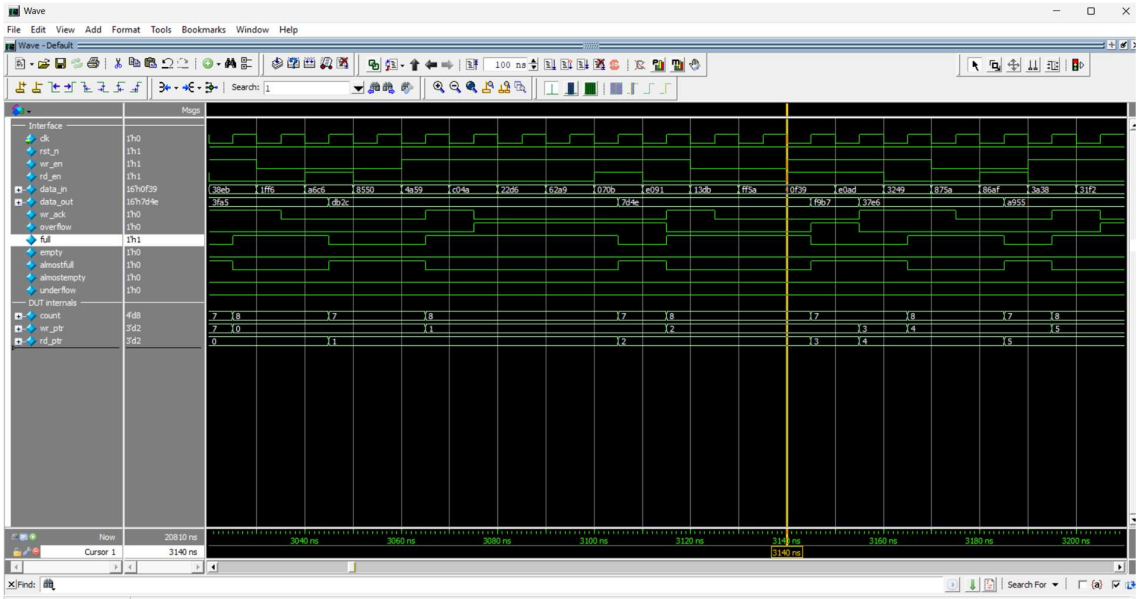
Random phase: constrained random wr_en / rd_en / data_in drive the FIFO through all occupancy levels.

Asynchronous reset while the FIFO holds data



Asynchronous reset: outputs and pointers clear at once when rst_n falls.

Simultaneous wr_en = 1 and rd_en = 1 - FIFO partly full / full

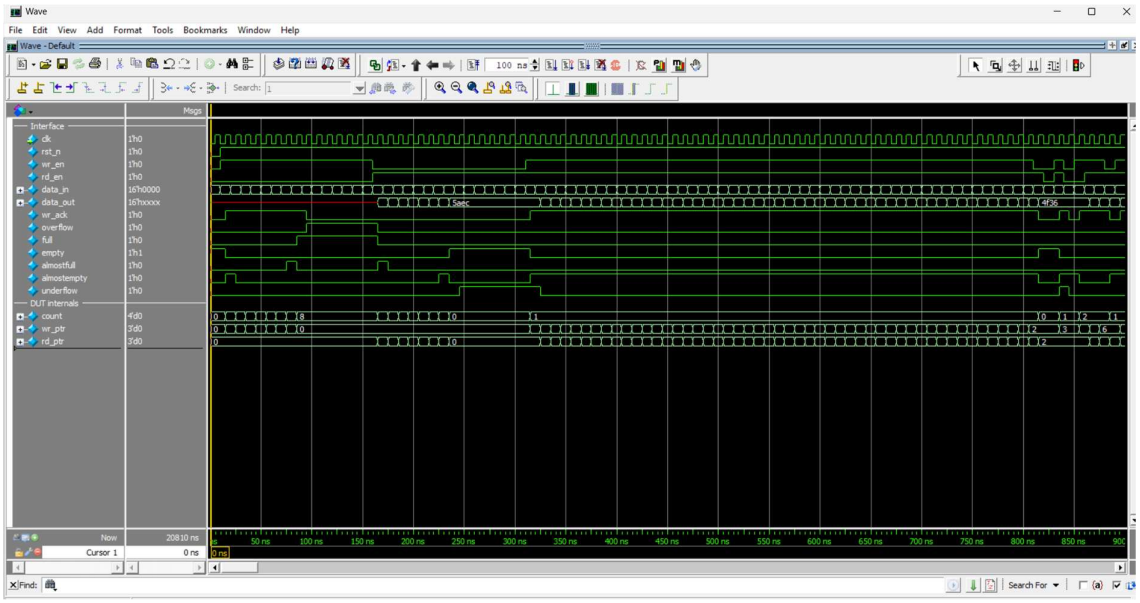


Simultaneous write and read on a full FIFO: only the read takes place and count decreases.

5.6 Overview of the whole main sequence

Zooming out shows the four sequences one after the other: reset, write-only (count rises to 8), read-only (count falls to 0), the 50 simultaneous items (count flat at 1) and the random phase.

Complete run



The main sequence: reset, write-only, read-only, simultaneous write/read, then random traffic.

6. Code Coverage Report

Code coverage report

```
================================================================================
=== Instance: /\top#DUT
=== Design Unit: work.FIFO
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        23        23         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\top#DUT

    Line         Item                    Count     Source
    ----         ----                    -----     ------
  File FIFO.sv
------------------------------------IF Branch------------------------------------
    12                                    2172     Count coming in to IF
    12              1                      186         if (!f_if.rst_n) begin
    17              1                     1062         else if (f_if.wr_en && count < FIFO_DEPTH) begin
    23              1                      924         else begin
Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    25                                     924     Count coming in to IF
    25              1                      367           if (f_if.full & f_if.wr_en)
    27              1                      557           else
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    34                                    2172     Count coming in to IF
    34              1                      186         if (!f_if.rst_n) begin
    38              1                      579         else if (f_if.rd_en && count != 0) begin
    43              1                     1407         else begin
Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    44                                    1407     Count coming in to IF
    44              1                       61           if (f_if.empty & f_if.rd_en)
    46              1                     1346           else
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    53                                    1869     Count coming in to IF
    53              1                      184         if (!f_if.rst_n) begin
    56              1                     1685         else begin
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    57                                    1685     Count coming in to IF
    57              1                      754           if ( ({f_if.wr_en, f_if.rd_en} == 2'b10 && !f_if.full) ||
    60              1                      271           else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01 && !f_if.empty) ||
                                           660     All False Count
Branch totals: 3 hits of 3 branches = 100.00%

------------------------------------IF Branch------------------------------------
    67                                    1115     Count coming in to IF
    67              1                      175       assign f_if.full = (count == FIFO_DEPTH) ? 1 : 0;
    67              2                      940       assign f_if.full = (count == FIFO_DEPTH) ? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    68                                    1115     Count coming in to IF
    68              1                      107       assign f_if.empty = (count == 0) ? 1 : 0;
    68              2                     1008       assign f_if.empty = (count == 0) ? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    69                                    1115     Count coming in to IF
    69              1                      219       assign f_if.almostfull = (count == FIFO_DEPTH-1) ? 1 : 0;
    69              2                      896       assign f_if.almostfull = (count == FIFO_DEPTH-1) ? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%

------------------------------------IF Branch------------------------------------
    70                                    1115     Count coming in to IF
    70              1                      125       assign f_if.almostempty = (count == 1) ? 1 : 0;
    70              2                      990       assign f_if.almostempty = (count == 1) ? 1 : 0;
Branch totals: 2 hits of 2 branches = 100.00%


Condition Coverage:
    Enabled Coverage              Bins   Covered    Misses  Coverage
    ----------------              ----   -------    ------  --------
    Conditions                      20        18         2    90.00%
```

```
================================Statement Details================================

Statement Coverage for instance /\top#DUT  --

    Line         Item                    Count     Source
    ----         ----                    -----     ------
  File FIFO.sv
    1                                              module FIFO (FIFO_if.DUT f_if);
    2                                                parameter FIFO_WIDTH = 16;
    3                                                parameter FIFO_DEPTH = 8;
    4                                                localparam max_fifo_addr = $clog2(FIFO_DEPTH);
    5
    6                                                reg [FIFO_WIDTH-1:0] mem [FIFO_DEPTH-1:0];
    7                                                reg [max_fifo_addr-1:0] wr_ptr, rd_ptr;
    8                                                reg [max_fifo_addr:0] count;
    9
    10                                               // Sequential Write & Overflow Logic
    11              1                     2172       always @(posedge f_if.clk or negedge f_if.rst_n) begin
    12                                                 if (!f_if.rst_n) begin
    13              1                      186           wr_ptr <= 0;
    14              1                      186           f_if.wr_ack <= 0;
    15              1                      186           f_if.overflow <= 0;
    16                                                 end
    17                                                 else if (f_if.wr_en && count < FIFO_DEPTH) begin
    18              1                     1062           mem[wr_ptr] <= f_if.data_in;
    19              1                     1062           f_if.wr_ack <= 1;
    20              1                     1062           f_if.overflow <= 0;
    21              1                     1062           wr_ptr <= wr_ptr + 1;
    22                                                 end
    23                                                 else begin
    24              1                      924           f_if.wr_ack <= 0;
    25                                                   if (f_if.full & f_if.wr_en)
    26              1                      367             f_if.overflow <= 1;
    27                                                   else
    28              1                      557             f_if.overflow <= 0;
    29                                                 end
    30                                               end
    31
    32                                               // Sequential Read & Underflow Logic
    33              1                     2172       always @(posedge f_if.clk or negedge f_if.rst_n) begin
    34                                                 if (!f_if.rst_n) begin
    35              1                      186           rd_ptr <= 0;
    36              1                      186           f_if.underflow <= 0;
    37                                                 end
    38                                                 else if (f_if.rd_en && count != 0) begin
    39              1                      579           f_if.data_out <= mem[rd_ptr];
    40              1                      579           f_if.underflow <= 0;
    41              1                      579           rd_ptr <= rd_ptr + 1;
    42                                                 end
    43                                                 else begin
    44                                                   if (f_if.empty & f_if.rd_en)
    45              1                       61             f_if.underflow <= 1;
    46                                                   else
    47              1                     1346             f_if.underflow <= 0;
    48                                                 end
    49                                               end
    50
    51                                               // Counter Logic handling simultaneous read/writes
    52              1                     1869       always @(posedge f_if.clk or negedge f_if.rst_n) begin
    53                                                 if (!f_if.rst_n) begin
    54              1                      184           count <= 0;
    55                                                 end
    56                                                 else begin
    57                                                   if ( ({f_if.wr_en, f_if.rd_en} == 2'b10 && !f_if.full) ||
    58                                                        ({f_if.wr_en, f_if.rd_en} == 2'b11 && f_if.empty) )
    59              1                      754             count <= count + 1;
    60                                                   else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01 && !f_if.empty) ||
    61                                                             ({f_if.wr_en, f_if.rd_en} == 2'b11 && f_if.full) )
    62              1                      271             count <= count - 1;
    63                                                 end
    64                                               end
    65
    66                                               // Combinational Flags
    67              1                     1116       assign f_if.full = (count == FIFO_DEPTH) ? 1 : 0;
    68              1                     1116       assign f_if.empty = (count == 0) ? 1 : 0;
    69              1                     1116       assign f_if.almostfull = (count == FIFO_DEPTH-1) ? 1 : 0;
    70              1                     1116       assign f_if.almostempty = (count == 1) ? 1 : 0;

Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         20        20         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\top#DUT  --

                                      Node      1H->0L      0L->1H  "Coverage"
                                      ----------------------------------------
                                  count[3-0]          1           1      100.00
                                 rd_ptr[2-0]          1           1      100.00
                                 wr_ptr[2-0]          1           1      100.00

Total Node Count     =         10
Toggled Node Count   =         10
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (20 of 20 bins)
```

```
----------------------------------------------------------------------
/\top#DUT /fifo_assert_inst/assert__p_wr_rd_mid
                    fifo_assertions.sv(48)              0          1
/\top#DUT /fifo_assert_inst/assert__p_wr_rd_full
                    fifo_assertions.sv(47)              0          1
/\top#DUT /fifo_assert_inst/assert__p_wr_rd_empty
                    fifo_assertions.sv(46)              0          1
/\top#DUT /fifo_assert_inst/assert__p_no_ack_when_full
                    fifo_assertions.sv(45)              0          1
/\top#DUT /fifo_assert_inst/assert__p_rd_wrap
                    fifo_assertions.sv(44)              0          1
/\top#DUT /fifo_assert_inst/assert__p_threshold
                    fifo_assertions.sv(43)              0          1
/\top#DUT /fifo_assert_inst/assert__p_wr_wrap
                    fifo_assertions.sv(42)              0          1
/\top#DUT /fifo_assert_inst/assert__p_almostempty
                    fifo_assertions.sv(41)              0          1
/\top#DUT /fifo_assert_inst/assert__p_almostfull
                    fifo_assertions.sv(40)              0          1
/\top#DUT /fifo_assert_inst/assert__p_full
                    fifo_assertions.sv(39)              0          1
/\top#DUT /fifo_assert_inst/assert__p_empty
                    fifo_assertions.sv(38)              0          1
/\top#DUT /fifo_assert_inst/assert__p_underflow
                    fifo_assertions.sv(37)              0          1
/\top#DUT /fifo_assert_inst/assert__p_overflow
                    fifo_assertions.sv(36)              0          1
/\top#DUT /fifo_assert_inst/assert__p_wr_ack
                    fifo_assertions.sv(35)              0          1
/\top#DUT /fifo_assert_inst/a_reset_count
                    fifo_assertions.sv(11)              0          1
/\top#DUT /fifo_assert_inst/a_reset_wr_ptr
                    fifo_assertions.sv(12)              0          1
/\top#DUT /fifo_assert_inst/a_reset_rd_ptr
                    fifo_assertions.sv(13)              0          1
/\top#DUT /fifo_assert_inst/a_reset_flags
                    fifo_assertions.sv(14)              0          1

Total Coverage By Instance (filtered view): 98.33%
```

Code coverage of FIFO.sv collected with the UVM environment.

7. Functional Coverage Report

```
Coverage Report by instance with details

================================================================================
=== Instance: /fifo_coverage_pkg
=== Design Unit: work.fifo_coverage_pkg
================================================================================

Covergroup Coverage:
    Covergroups                 1        na        na   100.00%
        Coverpoints/Crosses    16        na        na        na
            Covergroup Bins    66        66         0   100.00%
--------------------------------------------------------------------------------
Covergroup                                          Metric       Goal       Bins    Status

--------------------------------------------------------------------------------
 TYPE /fifo_coverage_pkg/fifo_coverage/cvr_gp      100.00%        100          -    Covered
    covered/total bins:                                66         66          -
    missing/total bins:                                 0         66          -
    % Hit:                                        100.00%        100          -
    Coverpoint wr_en_cp                           100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                   579          1          -    Covered
        bin auto[1]                                  1502          1          -    Covered
    Coverpoint rd_en_cp                           100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1415          1          -    Covered
        bin auto[1]                                   666          1          -    Covered
    Coverpoint full_cp                            100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1540          1          -    Covered
        bin auto[1]                                   541          1          -    Covered
    Coverpoint empty_cp                           100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1906          1          -    Covered
        bin auto[1]                                   175          1          -    Covered
    Coverpoint almostfull_cp                      100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1740          1          -    Covered
        bin auto[1]                                   341          1          -    Covered
    Coverpoint almostempty_cp                     100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1840          1          -    Covered
        bin auto[1]                                   241          1          -    Covered
    Coverpoint overflow_cp                        100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1714          1          -    Covered
        bin auto[1]                                   367          1          -    Covered
    Coverpoint underflow_cp                       100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  2020          1          -    Covered
        bin auto[1]                                    61          1          -    Covered
    Coverpoint wr_ack_cp                          100.00%        100          -    Covered
        covered/total bins:                             2          2          -
        missing/total bins:                             0          2          -
        % Hit:                                    100.00%        100          -
        bin auto[0]                                  1019          1          -    Covered
        bin auto[1]                                  1062          1          -    Covered
```

Project: UVM Synchronous FIFO

Functional coverage report - summary

```
    missing/total bins:                          0         6         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[0]>            470         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            196         1         -    Covered
        bin <auto[1],auto[0],auto[1]>            438         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            594         1         -    Covered
        bin <auto[0],auto[0],auto[1]>            103         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            280         1         -    Covered
    Illegal and Ignore Bins:
        ignore_bin full_with_read                  0                   -    ZERO
Cross cross_empty                              100.00%       100         -    Covered
    covered/total bins:                          8         8         -
    missing/total bins:                          0         8         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>             19         1         -    Covered
        bin <auto[0],auto[1],auto[1]>             46         1         -    Covered
        bin <auto[1],auto[0],auto[1]>             54         1         -    Covered
        bin <auto[0],auto[0],auto[1]>             56         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            451         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            150         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            978         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            327         1         -    Covered
Cross cross_almostfull                         100.00%       100         -    Covered
    covered/total bins:                          8         8         -
    missing/total bins:                          0         8         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>            170         1         -    Covered
        bin <auto[0],auto[1],auto[1]>             44         1         -    Covered
        bin <auto[1],auto[0],auto[1]>             71         1         -    Covered
        bin <auto[0],auto[0],auto[1]>             56         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            300         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            152         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            961         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            327         1         -    Covered
Cross cross_almostempty                        100.00%       100         -    Covered
    covered/total bins:                          8         8         -
    missing/total bins:                          0         8         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>            126         1         -    Covered
        bin <auto[0],auto[1],auto[1]>             18         1         -    Covered
        bin <auto[1],auto[0],auto[1]>             68         1         -    Covered
        bin <auto[0],auto[0],auto[1]>             29         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            344         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            178         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            964         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            354         1         -    Covered
Cross cross_overflow                           100.00%       100         -    Covered
    covered/total bins:                          6         6         -
    missing/total bins:                          0         6         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>            104         1         -    Covered
        bin <auto[1],auto[0],auto[1]>            263         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            366         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            196         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            769         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            383         1         -    Covered
    Illegal and Ignore Bins:
        ignore_bin overflow_without_write          0                   -    ZERO
Cross cross_underflow                          100.00%       100         -    Covered
    covered/total bins:                          6         6         -
    missing/total bins:                          0         6         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>             39         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            431         1         -    Covered
        bin <auto[0],auto[1],auto[1]>             22         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            174         1         -    Covered
        bin <auto[1],auto[0],auto[0]>           1032         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            383         1         -    Covered
    Illegal and Ignore Bins:
        ignore_bin underflow_without_read          0                   -    ZERO
Cross cross_wr_ack                             100.00%       100         -    Covered
    covered/total bins:                          6         6         -
    missing/total bins:                          0         6         -
    % Hit:                                 100.00%       100         -
    Auto, Default and User Defined Bins:
        bin <auto[1],auto[1],auto[1]>            347         1         -    Covered
        bin <auto[1],auto[0],auto[1]>            715         1         -    Covered
        bin <auto[1],auto[1],auto[0]>            123         1         -    Covered
        bin <auto[0],auto[1],auto[0]>            196         1         -    Covered
        bin <auto[1],auto[0],auto[0]>            317         1         -    Covered
        bin <auto[0],auto[0],auto[0]>            383         1         -    Covered
    Illegal and Ignore Bins:
        ignore_bin ack_without_write               0                   -    ZERO

COVERGROUP COVERAGE:
```

Functional coverage: all coverpoints and crosses are covered.

8. Sequential Domain Coverage Report

Sequential domain coverage - assertions

```
================================================================================
=== Instance: /top/DUT/fifo_assert_inst
=== Design Unit: work.fifo_assertions
================================================================================

Assertion Coverage:
    Assertions                      18        18         0   100.00%
--------------------------------------------------------------------
Name                 File(Line)                   Failure      Pass
                                                  Count        Count
--------------------------------------------------------------------
/top/DUT/fifo_assert_inst/assert__p_wr_rd_mid
                     fifo_assertions.sv(48)              0          1
/top/DUT/fifo_assert_inst/assert__p_wr_rd_full
                     fifo_assertions.sv(47)              0          1
/top/DUT/fifo_assert_inst/assert__p_wr_rd_empty
                     fifo_assertions.sv(46)              0          1
/top/DUT/fifo_assert_inst/assert__p_no_ack_when_full
                     fifo_assertions.sv(45)              0          1
/top/DUT/fifo_assert_inst/assert__p_rd_wrap
                     fifo_assertions.sv(44)              0          1
/top/DUT/fifo_assert_inst/assert__p_threshold
                     fifo_assertions.sv(43)              0          1
/top/DUT/fifo_assert_inst/assert__p_wr_wrap
                     fifo_assertions.sv(42)              0          1
/top/DUT/fifo_assert_inst/assert__p_almostempty
                     fifo_assertions.sv(41)              0          1
/top/DUT/fifo_assert_inst/assert__p_almostfull
                     fifo_assertions.sv(40)              0          1
/top/DUT/fifo_assert_inst/assert__p_full
                     fifo_assertions.sv(39)              0          1
/top/DUT/fifo_assert_inst/assert__p_empty
                     fifo_assertions.sv(38)              0          1
/top/DUT/fifo_assert_inst/assert__p_underflow
                     fifo_assertions.sv(37)              0          1
/top/DUT/fifo_assert_inst/assert__p_overflow
                     fifo_assertions.sv(36)              0          1
/top/DUT/fifo_assert_inst/assert__p_wr_ack
                     fifo_assertions.sv(35)              0          1
/top/DUT/fifo_assert_inst/a_reset_count
                     fifo_assertions.sv(11)              0          1
/top/DUT/fifo_assert_inst/a_reset_wr_ptr
                     fifo_assertions.sv(12)              0          1
/top/DUT/fifo_assert_inst/a_reset_rd_ptr
                     fifo_assertions.sv(13)              0          1
/top/DUT/fifo_assert_inst/a_reset_flags
                     fifo_assertions.sv(14)              0          1
```

Assertion coverage: every assertion was checked and none failed.

Sequential domain coverage - cover directives

```
Directive Coverage:
    Directives                    13        13         0   100.00%

DIRECTIVE COVERAGE:
--------------------------------------------------------------------------------------------
Name                                  Design Design   Lang File(Line)      Hits Status
                                      Unit   UnitType
--------------------------------------------------------------------------------------------
/top/DUT/fifo_assert_inst/cover__p_wr_rd_mid
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(63)
                                                                          296 Covered
/top/DUT/fifo_assert_inst/cover__p_wr_rd_full
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(62)
                                                                          101 Covered
/top/DUT/fifo_assert_inst/cover__p_wr_rd_empty
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(61)
                                                                           37 Covered
/top/DUT/fifo_assert_inst/cover__p_no_ack_when_full
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(60)
                                                                          351 Covered
/top/DUT/fifo_assert_inst/cover__p_rd_wrap
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(59)
                                                                           37 Covered
/top/DUT/fifo_assert_inst/cover__p_wr_wrap
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(58)
                                                                           86 Covered
/top/DUT/fifo_assert_inst/cover__p_almostempty
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(57)
                                                                          234 Covered
/top/DUT/fifo_assert_inst/cover__p_almostfull
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(56)
                                                                          326 Covered
/top/DUT/fifo_assert_inst/cover__p_full  fifo_assertions Verilog  SVA  fifo_assertions.sv(55)
                                                                          514 Covered
/top/DUT/fifo_assert_inst/cover__p_empty fifo_assertions Verilog  SVA  fifo_assertions.sv(54)
                                                                          170 Covered
/top/DUT/fifo_assert_inst/cover__p_underflow
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(53)
                                                                           59 Covered
/top/DUT/fifo_assert_inst/cover__p_overflow
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(52)
                                                                          351 Covered
/top/DUT/fifo_assert_inst/cover__p_wr_ack
                                      fifo_assertions Verilog  SVA  fifo_assertions.sv(51)
                                                                         1008 Covered
```

Cover directives: every scenario was reached by the UVM sequences.

9. Bug Report

The first four bugs were found in the original RTL during the SystemVerilog project and are fixed in the FIFO.sv used here.

Bug 1: underflow signal is combinational

Description: the requirements state that underflow must be a sequential output, but the original RTL used a continuous assignment.

```
assign underflow = (empty && rd_en)? 1 : 0;
```

Fixed: the assignment was removed and underflow is generated in the sequential read block, so it updates on the clock edge.

```
  // Sequential Read & Underflow Logic
  always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
      rd_ptr <= 0;
      f_if.underflow <= 0;
    end
    else if (f_if.rd_en && count != 0) begin
      f_if.data_out <= mem[rd_ptr];
      f_if.underflow <= 0;
      rd_ptr <= rd_ptr + 1;
    end
    else begin
      if (f_if.empty & f_if.rd_en)
        f_if.underflow <= 1;
      else
        f_if.underflow <= 0;
    end
  end
```

Bug 2: incorrect almost-full logic

Description: almost full means that only one more write can be performed, so it must be asserted at FIFO_DEPTH-1. The original RTL used FIFO_DEPTH-2.

```
assign almostfull = (count == FIFO_DEPTH-2)? 1 : 0;
```

Fixed: Changed the combinational condition from FIFO_DEPTH-2 to FIFO_DEPTH-1.

```
assign f_if.almostfull = (count == FIFO_DEPTH-1) ? 1 : 0;
```

Bug 3: missing asynchronous reset of wr_ack and overflow

Bug Description: The sequential outputs wr_ack and overflow were not initialized inside the active-low asynchronous reset block (!rst_n), which can lead to unknown (X) states at startup.

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
```

```
        wr_ptr <= 0;
    end
```

Fixed: Added f_if.wr_ack <= 0; and f_if.overflow <= 0; inside the reset condition.

```
 always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
      wr_ptr <= 0;
      f_if.wr_ack <= 0;
      f_if.overflow <= 0;
    end
```

Bug 4: simultaneous read/write counter logic

Bug Description: The original count logic failed to handle edge cases when both wr_en and rd_en are high simultaneously. Specifically, if both are high when the FIFO is empty, the count should increment (only write happens). If full, the count should decrement (only read happens).

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= 0;
    end
    else begin
        if  ( ({wr_en, rd_en} == 2'b10) && !full)
            count <= count + 1;
        else if ( ({wr_en, rd_en} == 2'b01) && !empty)
            count <= count - 1;
    end
end
```

Fixed: Rewrote the counter block to explicitly handle simultaneous reads and writes when the FIFO is either completely empty or completely full.

```
 // Counter Logic handling simultaneous read/writes
  always @(posedge f_if.clk or negedge f_if.rst_n) begin
    if (!f_if.rst_n) begin
      count <= 0;
    end
    else begin
      if ( ({f_if.wr_en, f_if.rd_en} == 2'b10 && !f_if.full) ||
           ({f_if.wr_en, f_if.rd_en} == 2'b11 && f_if.empty) )
        count <= count + 1;
      else if ( ({f_if.wr_en, f_if.rd_en} == 2'b01 && !f_if.empty) ||
                ({f_if.wr_en, f_if.rd_en} == 2'b11 && f_if.full) )
        count <= count - 1;
    end
  end
```

Project: UVM Synchronous FIFO

Final result of the UVM run(Transcript)

Final result: the scoreboard summary shows no mismatches and the UVM report summary shows zero errors.

```
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_NO_DEPRECATED undefined.
#   See http://www.eda.org/svdb/view.php?id=3313 for more details.
#
#   You are using a version of the UVM library that has been compiled
#   with `UVM_OBJECT_MUST_HAVE_CONSTRUCTOR undefined.
#   See http://www.eda.org/svdb/view.php?id=3770 for more details.
#
#       (Specify +UVM_NO_RELNOTES to turn off this notice)
#
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(278) @ 0: reporter [Questa UVM]  questa_uvm::init(all)
# UVM_INFO @ 0: reporter [RNTST] Running test fifo_test...
# UVM_INFO fifo_test.sv(35) @ 0: uvm_test_top [TEST] Starting FIFO Main Sequence...
# ****************************************************************************
# * Questa UVM Transaction Recording Turned ON.                              *
# * recording_detail has been set.                                           *
# *  To turn off, set 'recording_detail' to off:                             *
# * uvm_config_db#(int)            ::set(null, "", "recording_detail", 0); *
# * uvm_config_db#(uvm_bitstream_t)::set(null, "", "recording_detail", 0); *
# ****************************************************************************
# UVM_INFO fifo_test.sv(37) @ 20810: uvm_test_top [TEST] Finished FIFO Main Sequence.
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1267) @ 20810: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO fifo_scoreboard.sv(74) @ 20810: uvm_test_top.fifoenv.sb [SB_REPORT] Scoreboard summary: correct = 579, errors = 0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO :    7
# UVM_WARNING :    0
# UVM_ERROR :    0
# UVM_FATAL :    0
# ** Report counts by id
# [Questa UVM]     2
# [RNTST]     1
# [SB_REPORT]     1
# [TEST]     2
# [TEST_DONE]     1
# ** Note: $finish    : E:/QuestaSim/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
#    Time: 20810 ns  Iteration: 66  Instance: /top
# Break in Task uvm_pkg/uvm_root::run_test at E:/QuestaSim/win64/../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
```

10. Do Files and How to Run

The project is run in two steps. Open QuestaSim, change the directory to the project folder (File > Change Directory) and run:

1. `do run.do` - compiles every file listed in `src_files.list`, starts the simulation (`-coverage -classdebug -uvmcontrol=all`), adds the waves and runs until UVM calls `$finish`.
2. `do save_cov.do` - saves the two databases, closes the simulation and writes the three text reports.

run.do

```
vlib work
vmap work work
vlog +cover -f src_files.list
vsim -coverage -voptargs=+acc work.top -classdebug -uvmcontrol=all +UVM_TR_RECORD
view wave
add wave -divider {Interface}
add wave /top/f_if/*
add wave -divider {DUT internals}
add wave -radix unsigned /top/DUT/count /top/DUT/wr_ptr /top/DUT/rd_ptr
run -all
wave zoom full
```

save_cov.do

```
# ========================================
# Step 2 : after the run breaks at $finish,
#          save the databases and write reports
#   usage:  do save_cov.do
# ========================================
# full database: design code coverage + covergroups + assertions
coverage save fifo_tb.ucdb
# design-only database (code coverage of the DUT, no testbench noise)
coverage save fifo_code.ucdb -du FIFO
quit -sim
# ---------- report 1 : code coverage (design only) ----------
vcover report fifo_code.ucdb -details -annotate -all-output code_coverage_rpt.txt
# ---------- report 2 : functional coverage (covergroups) ----------
vcover report fifo_tb.ucdb -cvg -details -output functional_coverage_rpt.txt
# ---------- report 3 : sequential domain coverage (assertions + cover directives) ----------
vcover report fifo_tb.ucdb -assert -directive -details -output assertion_coverage_rpt.txt
```

Report file	Created by	Content
code_coverage_rpt.txt	vcover report fifo_code.ucdb -details -annotate -all	code coverage of FIFO.sv (design only)
functional_coverage_rpt.txt	vcover report fifo_tb.ucdb -cvg -details	covergroup cvr_gp: coverpoints, crosses, bins
assertion_coverage_rpt.txt	vcover report fifo_tb.ucdb -assert -directive -details	sequential domain coverage: assertions and cover directives